

ARCHITECTURE EXPLANATION

Adaptive Smart Cabin & Vehicle Health Monitoring System

How the system is structured, how the modules connect, and how data flows from sensor to screen

1. What Is Architecture — And Why It Matters

Architecture means how a system is organised. It answers three questions: What are the pieces? What does each piece do? How do the pieces talk to each other?

In this project the architecture was designed to mirror a real car ECU (Electronic Control Unit) that runs Adaptive AUTOSAR software. That means the code is split into layers — just like a real car — and each layer only talks to the layer below it. This makes the system easy to change (fix one module without breaking the rest) and easy to test (test one module in isolation).

Think of it like a factory. The sensor floor makes parts (sensor readings). The assembly floor checks quality (alert manager). The records office files reports (logger). The front office shows the status board (dashboard). Each floor does one job and hands work to the next.

2. The Four-Module File Structure

The entire project lives in exactly four header/source pairs. The evaluators expected this exact layout:

```
Project/
├── include/
│   ├── Sensor.hpp      ← Module 1: sensor definitions
│   ├── Alert.hpp       ← Module 2: alert + DTC + alert manager
│   ├── Logger.hpp      ← Module 3: logger + statistics + config
│   └── Dashboard.hpp   ← Module 4: display + threads + ECU + OTA + CAN + SOA
├── src/
│   ├── Sensor.cpp      ← Module 1 implementation
│   ├── Alert.cpp       ← Module 2 implementation
│   ├── Logger.cpp      ← Module 3 implementation
│   ├── Dashboard.cpp   ← Module 4 implementation
│   └── main.cpp        ← Composition root: wires everything + runs threads
├── logs/vehicle_log.txt ← created at runtime
├── data/config.txt      ← settings file
└── Makefile            ← build instructions
```

Each header file is a group — it holds all the classes that belong to one logical concern. The .cpp file for each header contains the actual code that makes those classes work.

3. The Compile-Time Include Chain

When the compiler builds the project, it needs to know about classes before it can use them. This is controlled by which file includes which. There is a strict rule in this project: the includes form a one-way chain with no loops.

3.1 The Chain

```
Sensor.hpp    – includes NOTHING from this project (foundation)
  ↑ included by
Alert.hpp     – does NOT include Sensor.hpp (doesn't need it)
               – forward-declares 'class Logger' to avoid a loop
  ↑ included by
Logger.hpp    – includes Alert.hpp (needs Alert for logAlert function)
  ↑ included by
Dashboard.hpp – includes Logger.hpp AND Sensor.hpp (needs both)
  ↑ included by
main.cpp      – includes all four headers (it's the top-level wiring point)
```

3.2 Why No Loops

If Alert.hpp included Logger.hpp, and Logger.hpp included Alert.hpp, the compiler would try to read each file while reading the other — an infinite loop. The solution is the forward declaration in Alert.hpp:

```
// In Alert.hpp:
class Logger;    // just tells the compiler 'Logger exists somewhere'
                // does NOT include Logger.hpp
```

AlertManager holds a shared_ptr<Logger>. A pointer only needs to know the class exists — not what's inside it. The full #include "Logger.hpp" happens only in Alert.cpp where Logger's functions are actually called. This is called 'forward declaration' and it's a key technique for breaking circular dependencies.

Rule: headers forward-declare when possible. .cpp files do the actual #include. This keeps compile times fast and avoids circular dependency crashes.

4. The Four-Layer Architecture

The system is built in layers, each sitting on top of the one below. This is identical to how Adaptive AUTOSAR organises real automotive software.

```
LAYER 4:  Dashboard / Infrastructure Layer
          (Dashboard.hpp / Dashboard.cpp)
          Shows results to the driver. Manages all threads.
          Uses: Sensor layer + Alert layer + Logger layer

LAYER 3:  Logger / Statistics Layer
          (Logger.hpp / Logger.cpp)
          Records events to disk. Tracks stats. Reads config.
          Uses: Alert layer
```

```
LAYER 2: Alert Layer
        (Alert.hpp / Alert.cpp)
        Detects danger. Raises + stores + prioritises warnings.
        Uses: forward declaration of Logger only

LAYER 1: Sensor / Foundation Layer
        (Sensor.hpp / Sensor.cpp)
        Simulates the 6 car sensors. Produces values.
        Uses: NOTHING from this project
```

The important rule: a layer can only use layers BELOW it. Dashboard uses Sensor, Alert, and Logger. Logger uses Alert. Alert uses nothing. Sensor uses nothing. Nobody reaches upward. This prevents tangled dependencies.

5. What Each Module Contains

5.1 Module 1 — Sensor (Sensor.hpp / Sensor.cpp)

Component	What It Is	Purpose
SensorException	Custom error class	Thrown when a sensor fails
SensorTimeoutException	Inherits from SensorException	Specific error for timed-out sensors
SensorHealth (enum)	HEALTHY/DEGRADED/FAULT/TIMEOUT/OFFLINE	Tracks the state of each sensor
Sensor (abstract base)	The contract all sensors must follow	Defines 4 pure virtual functions
EngineTemperatureSensor	Concrete sensor class	Simulates engine temp 60-130°C
BatterySensor	Concrete sensor class	Simulates battery 7-15V
SpeedSensor	Concrete sensor class	Simulates speed 0-180 km/h
DoorSensor	Concrete sensor class	Simulates door OPEN/CLOSED
SeatbeltSensor	Concrete sensor class	Simulates seatbelt LOCKED/UNLOCKED
TirePressureSensor	Concrete sensor class	Simulates tire pressure 15-40 PSI

5.2 Module 2 — Alert (Alert.hpp / Alert.cpp)

Component	What It Is	Purpose
AlertSeverity (enum)	INFO/WARNING/CRITICAL/EMERGENCY	How serious the alert is
AlertCategory (enum)	SAFETY/POWERTRAIN/CHASSIS/BODY/...	Which car system raised it
Alert class	One warning notification	Stores source, message, severity, category, timestamp, id
DTC struct	One diagnostic fault code	Stores P-code, description, count, timestamps, active flag
DTCManager class	Stores all fault codes	setDTC/clearDTC, unordered_map<string,DTC>
AlertManager class	The alert brain	raiseAlert, clearAlert, dedup, priority sort, lambda filter

5.3 Module 3 — Logger (Logger.hpp / Logger.cpp)

Component	What It Is	Purpose
LoggerException	Custom error class	Thrown when log file can't be opened
LogLevel (enum)	DEBUG/INFO/WARN/ERROR/CRITICAL	Tags each log line with severity
Logger class	RAII file writer	Opens file in ctor, closes in dtor, flush after every write (crash-safe)
VehicleStatistics class	Rolling stats + graphs	Tracks max speed, avg temp, distance; draws 3 bar graphs
ConfigException	Custom error class	Thrown for bad config file
ConfigLoader class	INI file parser	Reads data/config.txt, key=value pairs, typed getters

5.4 Module 4 — Dashboard (Dashboard.hpp / Dashboard.cpp)

Component	What It Is	Purpose
VehicleMode (enum)	NORMAL/DEGRADED/LIMP_HOME/SAFE_STOP	Current operating state of the vehicle
ECUHealthMonitor class	Watchdog + limp-home controller	Checks sensor timeouts, manages vehicle mode, attempts recovery
OTAState (enum)	IDLE→CHECKING→...→SUCCESS	7 states of a firmware update
OTAManager class	OTA state machine	Advances firmware update state one step per tick
ShutdownToken class	Cooperative stop signal	atomic<bool> + condition_variable, wakes all threads instantly

ThreadManager class	RAII thread container	Spawns threads, joins all on destruction
DriverPreferences struct	Threshold values per driver	maxSpeed, overheat, lowBattery, lowTire thresholds
DriverProfile class	4 driver profiles	Default/Eco/Sport/Chauffeur with different thresholds
CANFrame struct	One CAN message	11-bit ID + 8-byte data + timestamp
CANBus class	CAN message bus	Thread-safe publish/receive queue of CANFrames
ServiceManager class	SOA communication layer	registerService/callService + subscribe/publish topics
Dashboard class	Cockpit display	Renders header, sensors, alerts, stats to any ostream

6. Runtime Object Ownership

When the program runs, `main.cpp` creates every object and connects them. The ownership model tells you who owns what and who just borrows it.

6.1 `shared_ptr` — many owners

A `shared_ptr` allows multiple parts of the program to hold a reference to the same object. The object is automatically deleted when the last `shared_ptr` to it is destroyed. In this project:

```
shared_ptr<Logger>           → shared by DTCManager, AlertManager,
                             ECUHealthMonitor, CANBus, OTAManager
                             (they all write to the same log file)

shared_ptr<DTCManager>       → shared by AlertManager, ECUHealthMonitor
                             (both auto-generate fault codes)

shared_ptr<AlertManager>     → shared by main monitor thread, ECUHealthMonitor,
                             Dashboard

shared_ptr<VehicleStatistics> → shared by main monitor thread, Dashboard

shared_ptr<ECUHealthMonitor> → shared by all 5 threads (heartbeat)
```

6.2 `unique_ptr` — one owner

```
unique_ptr<Dashboard>       → owned only by the dashboard thread
                             (only one place renders the dashboard)
```

6.3 `vector<shared_ptr<Sensor>>` — the sensor list

```
vector<shared_ptr<Sensor>> sensors
→ passed to: sensor thread (updates them)
             monitor thread (reads values)
             ECUHealthMonitor (watchdog checks)
             Dashboard (displays values)
```

All six sensors are stored as `shared_ptr<Sensor>` (the base type). This is what allows polymorphism — looping over the list and calling `update()` runs the correct version for each sensor type automatically.

Key rule: no raw pointers, no `new/delete`. Every object is managed by a smart pointer. This is RAI — when the smart pointer is destroyed, the object is automatically cleaned up.

7. The Complete Data Flow

This is how data travels from a sensor reading to what you see on screen. It goes through all four modules.

Step 1 — Sensor Thread updates the sensor (Module 1)

```
// Every 500ms in the Sensor Thread:
for (auto& s : sensors)
    s->update();

// Inside EngineTemperatureSensor::update():
lock_guard<mutex> lk(mutex_);    // lock
temperature_ += random(-2.0, 3.0); // change value
clamp(60, 130);                // keep realistic
touch();                        // stamp watchdog time
                                // unlock (auto, RAII)
```

Step 2 — Monitor Thread reads the sensor and checks thresholds (Modules 1+2+3)

```
// Every 600ms in the Monitor Thread:
auto prefs = driver->current(); // get driver thresholds

double t = engineSensor->getNumericValue(); // locked read
stats.recordEngineTemp(t);                 // update statistics

if (t > prefs.overheatAlert) { // 110°C for Default, 115 for Sport
    alertMgr.raiseAlert(Alert(
        "EngineTemp",
        "CRITICAL ENGINE OVERHEAT (" + value + ")",
        AlertSeverity::CRITICAL,
        AlertCategory::POWERTRAIN
    ));
} else {
    alertMgr.clearAlert("EngineTemp"); // cooled down, remove warning
}
```

Step 3 — AlertManager processes the alert (Module 2)

```
void AlertManager::raiseAlert(const Alert& alert) {
    bool isNew = false;
    {
        lock_guard<mutex> lk(mutex_); // lock
        if (not already alerting from this source) {
            active_[source] = alert; // add to map
            history_.push_back(alert); // add to history
            isNew = true;
        }
    }
}
```

```
    } // ← UNLOCK here (before slow operations)

    if (isNew) {
        logger->logAlert(alert);           // write to file (slow, outside lock)
        dtcManager->setDTC("P0309", ...); // register fault code
    }
}
```

Step 4 — Logger writes to disk and console (Module 3)

```
void Logger::write(LogLevel level, const string& msg) {
    lock_guard<mutex> lk(mutex_); // only one thread writes at a time
    ostringstream line;
    line << "[" << timestamp() << "]" "
        << "[" << setw(8) << toString(level) << "]" "
        << msg << "\n";

    stream_ << line.str(); // write to file
    stream_.flush();       // force to disk (crash-safe)
    cout << line.str();   // also print to terminal
}
```

Output to vehicle_log.txt:

```
[2026-05-27 10:54:31.045] [CRITICAL ] CRITICAL ENGINE OVERHEAT (112.3 C)
```

Step 5 — Dashboard Thread renders everything to screen (Module 4)

```
// Every 2000ms in the Dashboard Thread:
cout << *dashboard;

// This calls operator<< which calls render(cout)
void Dashboard::render(ostream& os) {
    renderHeader(os); // driver, mode, fault count
    renderSensors(os); // all 6 sensor values + health
    renderAlerts(os); // active alerts in priority order
    renderStats(os); // uptime, distance, graphs
    renderFooter(os);
}
```

The complete journey: sensor changes value → monitor reads it → alert is raised → logger writes to file and console → dashboard renders it to screen. All 4 modules participate in every alert cycle.

8. How The Modules Talk To Each Other

Each module is given references to the other modules it needs through the constructor (called 'dependency injection'). No module creates its own dependencies — they receive them from main.cpp. This makes each module testable in isolation.

8.1 Who passes what to whom

Module	Receives in constructor	Therefore can call
DTCManager	shared_ptr<Logger>	logger->log() when a DTC is set
AlertManager	shared_ptr<Logger>, shared_ptr<DTCManager>	logAlert(), setDTC() on new critical alerts
ECUHealthMonitor	sensors vector, AlertManager, Logger, DTCManager	raiseAlert(), setDTC(), log(), sensor->setHealth()
CANBus	shared_ptr<Logger>	log() every frame published
OTAManager	shared_ptr<Logger>	log() each state transition
Dashboard	sensors, AlertManager, Stats, ECU, DriverProfile	All read functions to build the display

8.2 The service-oriented communication layer

Beyond direct method calls, modules can also communicate through the ServiceManager without knowing each other directly. This mirrors Adaptive AUTOSAR's `ara::com`.

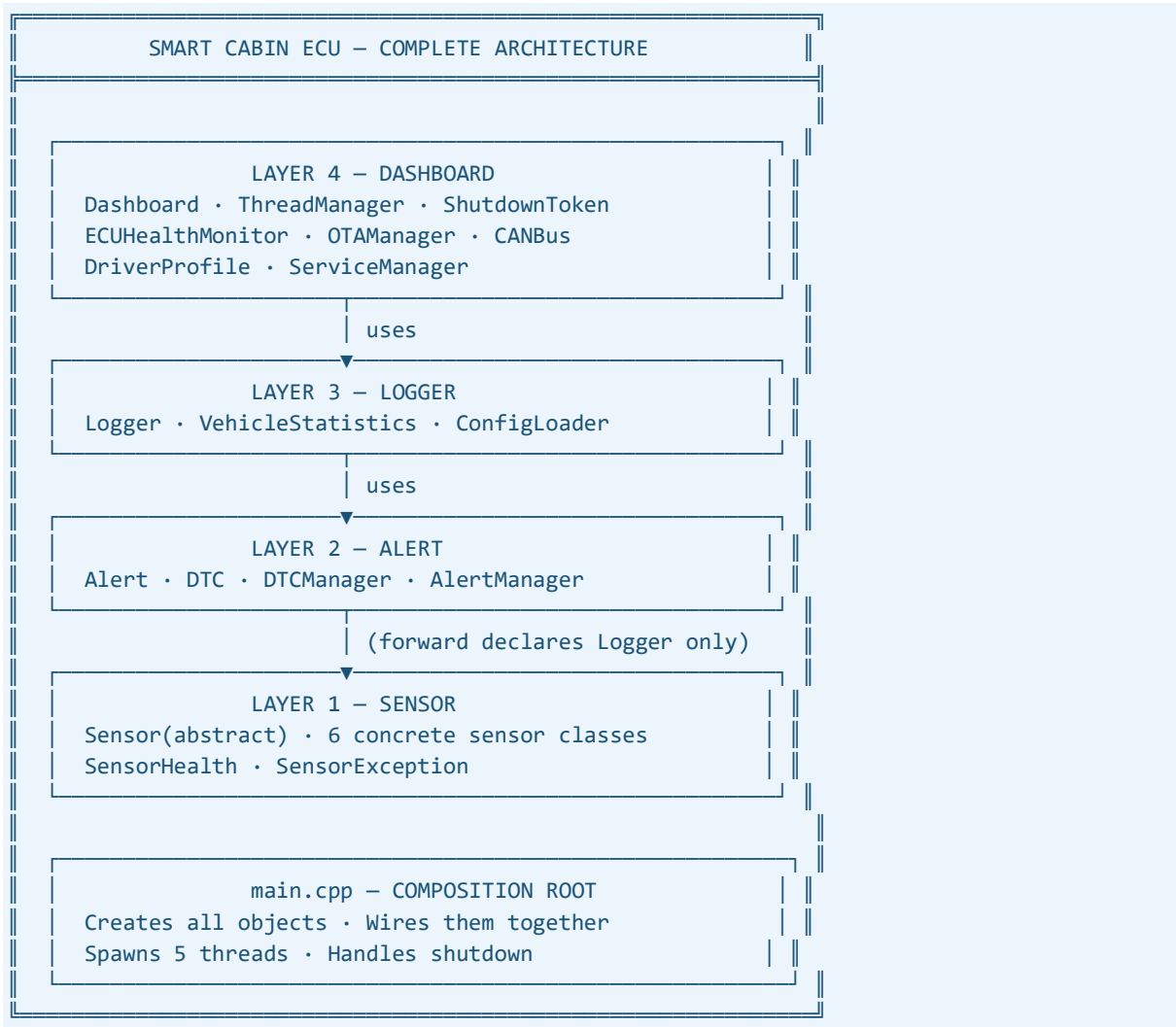
```
// In main.cpp – register services:
services->registerService("GetMode", [&](any) {
    return toString(ecuMonitor->getMode()); // 'NORMAL' or 'LIMP_HOME'
});

// Subscribe to events:
services->subscribe("alert.raised", [&](any payload) {
    stats->recordAlertRaised(); // count every alert
});

// In Monitor Thread – publish events:
for (auto& a : alertMgr->getActiveAlerts())
    services->publish("alert.raised", a);
```

The statistics module doesn't depend on the alert module directly. It just subscribes to an event topic. The alert module doesn't know statistics exists — it just publishes. This loose coupling is the SOA (Service-Oriented Architecture) feature.

9. Architecture Block Diagram



10. Automotive Mapping — Real-World Equivalents

Every module in this project has a direct real-world equivalent in Adaptive AUTOSAR. This is what makes the architecture authentic.

This Project	Real Automotive Component	AUTOSAR Standard
Sensor Framework (Module 1)	MCAL — Microcontroller Abstraction Layer	AUTOSAR Classic Platform
AlertManager (Module 2)	Vehicle Health Manager (VHM)	Adaptive AUTOSAR Platform

Dashboard (Module 4)	Instrument Cluster / HMI ECU	Cockpit Domain Controller
Logger (Module 3)	Diagnostic Event Manager (DEM)	AUTOSAR DEM SWC
DTCManager (Module 2)	OBD-II Diagnostic Trouble Codes	ISO 15765 / SAE J2012
ThreadManager + 5 threads	Execution Management	ara::exec
ShutdownToken	Execution State Machine	ara::exec::State
Mutex / lock_guard	Inter-process Synchronisation	AUTOSAR RTE
CANSimulator (Module 4)	CAN Communication Stack	AUTOSAR COM / PDUR
ServiceManager (Module 4)	SOME/IP Service-Oriented Comms	Adaptive AUTOSAR ara::com
ECUHealthMonitor (Module 4)	Platform Health Management + Watchdog	ara::phm
ConfigLoader (Module 3)	Parameter Manager / ARXML Config	AUTOSAR ECUC
OTAManager (Module 4)	Update & Configuration Management	Adaptive AUTOSAR UCM